

Principes de Architecture Parallele.

$$\begin{array}{l} S_1 : A = B + C \\ S_2 : D = E + F \end{array} \quad \boxed{\begin{array}{c} S_1 \parallel S_2 \\ ? \end{array}} \Rightarrow \begin{array}{l} A, D \text{ correct} \\ B, C, E, F \text{ OK} \end{array}$$

• Problème d'organisation matérielle : CPU/mémoire.

• Utilisation // ELisne

• Réseaux InterConnexion.

Pourquoi Archi // : Besoins en calcul.

Application ? ←

① → Jeu (GPU graphic processing unit.)

② → Simulation numérique

② → Bitcoin mining / Block chain.

③ → IA training / inference.

↳ matrice multiplication dense

CPU → Tensor product Unit ← spécialisation par GPU
TPU

$$\left. \begin{array}{l} S_1 : A = B + C \\ S_2 : D = E + F \\ \quad \quad \quad E \quad A \end{array} \right\} \begin{array}{l} S_1 \text{ puis } S_2 \\ \cancel{S_1 \parallel S_2} \end{array}$$

$$A = B + C \quad \parallel \quad D = (A) + F$$

$$\left\{ \begin{array}{l} \text{case de mémoire } m \\ m(A) \cap m(E) = \emptyset \\ m(A) \cap m(F) = \emptyset \\ m(D) \cap m(B) = \emptyset \\ m(D) \cap m(C) = \emptyset \end{array} \right.$$

$S_1 \xrightarrow{ACC} X$

$S_2 \xrightarrow{ACC} X$

$S_1 \backslash S_2$	Read	Write
Read	RAR	RAW
Write	WAR	WAW

Write After Write

$D = E + F$
 $D = H + C$

Ordre !

pas //

propager D par D_1

boucle L_1 : est-il peut paralleliser ?

DO $I = 1, N$

$S_1 = C(I) = A(I) + B(I)$

ENDDO

$S_1(1) \dots S_1(100)$

condition: $C(1:100) \cap A(1:100) = \emptyset$ disjoint l'écriture et lecture.

$C(1:100) \cap B(1:100)$

L_2 DO $I = 1, 100$

$C(I) = C(I-3) + B(I)$

ENDDO $C(I+3) + B(I)$

$B(1:100) \cap C(I-3, 100) = \emptyset$

Analyse :

Itération ^(I)	Write	Read	Read
1	$C(1)$	$C(I-2)$	$C(4)$
2	$C(2)$	$C(I-1)$	$C(5)$
3	$C(3)$	$C(0)$	$C(6)$
4	$C(4)$	$C(1)$	$C(7)$

Read after Read } $D = E + F$
 $H = E + C$ read en mm temps ✓

OK pour parallelism.

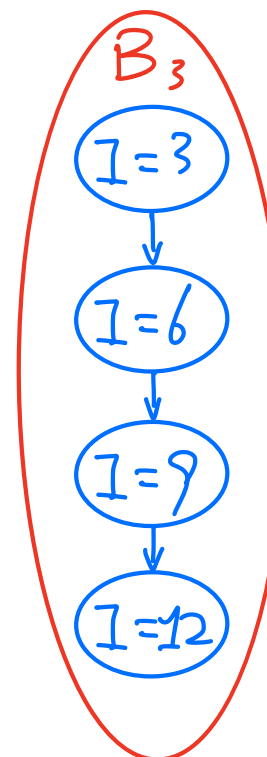
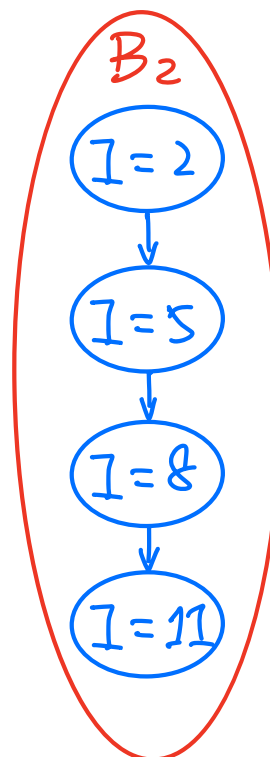
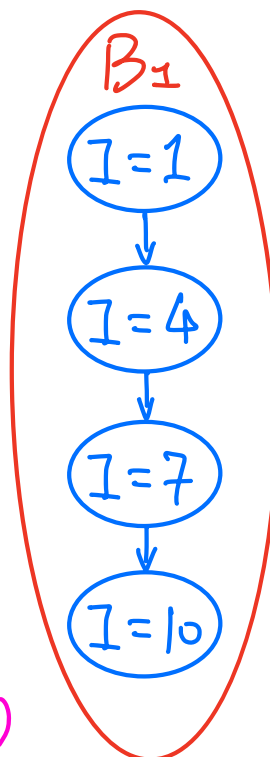
Read after Write } $D = E + F$ write D
 $H = D + C$ read D

Il faut respecter l'ordre : pas OK pour //

Write after Read } $H = D + C$ read D
 $D = E + F$

propager D par $D_1 = E + F$

Ordre ! pas //



5	c(5)	c(2)	c(8)
6	c(6)	c(3)	c(9)
7	c(7)	c(4)	c(10)
8	c(8)	c(5)	c(11)
9	c(9)	c(6)	c(12)

```

DO I = 1, 100 //
  C(I) = C(I+3) + B(I)
ENDDO

```

```

DO I = 1, 100
  C(I) = TMP(I)
ENDDO

```

```

DO I = 1, N
  A(g(I)) =
    = A(f(I))
ENDDO

```

$\exists ? (K, L) / 1 \leq K \leq N$
 $1 \leq L \leq N$

$K \in N$
 $L \in N$

$g(K) = f(L)$
 Test de dépendance

Si toutes les itérations soient
 exécutable en parallelism.

$K \leq L$.
 $\exists (K, L) g(K) = f(L)$
 RAW :
 $IT(K) \rightarrow IT(L)$

- ① même case est touché par une lecture et une écriture
- ② Savoir quel ordre

$K > L$.
 $\exists (K, L) g(K) = f(L)$
 WAR.
 $IT(L) \rightarrow IT(K)$

$$\begin{aligned}
 g(K) &= \alpha K + \beta \\
 f(L) &= \gamma L + \delta \\
 g(K) &= f(L) \\
 \alpha K + \beta &= \gamma L + \delta
 \end{aligned}$$

$$\text{PGCD}(\alpha, \delta) = p$$

$$\rightarrow \frac{\alpha K}{p} - \frac{\delta L}{p} = \frac{\delta - \beta}{p}$$

Test PGCD

DO I: 1, 100

$$A(2I+1) = A(2I)$$

$$2K+1 \stackrel{?}{=} 2L$$

$$p = 2$$

Généralisation :

$$\vec{f} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$$

$$\vec{g} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$$

DO I = 1, P

DO J = 1, Q

$$A(\vec{f}(I, J)) = A(\vec{g}(I, J))$$

ENDDO

ENDDO

$$\vec{f}(K, L) = \vec{g}(T, U)$$

$$(K, L) \propto (T, U)$$

l'ordre [↑]lexicographique $\rightarrow$ correspond à l'ordre d'exécution

Soit $K < T$ ou $K = T, L \leq U$

PGCD

$$\alpha K + \beta L + \delta = \varphi T + \Delta U + \psi$$

$$p = \text{PGCD}(\alpha, \beta, \varphi, \Delta) \quad p \mid (\psi - \delta)$$

diviser

Analyse de dépendances

Faite par le compilateur conservatif

Une boucle non parallélisable (dépendance) ne doit pas être parallélisée

Inéquation + équation de dépendance non résolues

$\Rightarrow$ Déclare dépendance $\Rightarrow$ non parallèle

Declarer non parallèle .

Une boucle qui est de fait parallèle . Le code est peu performant mais juste

DO I = 1, 5

 A[B(I)] = 3 * I
ENDDO

B(1) = 10

B(2) = 3

B(3) = 8

B(4) = 2

B(5) = ~~7~~

OK //

A(B(1)) = 3

DO I = 3, 5
 A(B(I)) = 3 * I

ENDDO

3

NON!

2, 4 //

1, 3, 5 //